# Session 10

## File Handling,Serialization & Deserialization

### Topic to be covered

```
1.Types of Data in Input/Output
2.How file I/O is done in most Programming Language
3.Writing to a file - TXT file .New file
4.Writing multiple Strings
5.Writing to a file - TXT file .Existing file
6.How open() works?
7.Problem with 'w' mode. Append mode is solution
8.Writing many lines
10.Reading from files
11.Reading from files readlines()
12.Reading big file line by line
13.Using context manager(with)
14.reading 10 char at a time till end file
15.Reading big file in chunks
16.seek and tail
17.Problem with text file
18.Working with binary files
19.Working with other data types in file handling
20.Serialization and Deserialization
21.Serialization using JSON
22.Deserialization using JSON .load()
23.Serialization and Deserialization of tuple
24.Serialization and Deserialization of nested dictionary
25.Serialization and Deserialization of custom object
26.Pickling
27.Pickle vs JSON
```

## 1.Types of data used for I/O:

- Text - '12345' as a sequence of unicode chars
- Binary - 12345 as a sequence of bytes of its binary equivalent

***Hence there are 2 file types to deal with***

- Text files - All program files are text files
- Binary Files - Images,music,video,exe files

## 2. How File I/O is done in most programming languages

- Open a file
- Read/Write data
- Close the file

### 3. Writing to a file - TXT file .New file

In [4]:
```
# case 1 - if file is not present
f = open('sample.txt','w')
f.write('Hello world')
```

Out[4]: 11

In [2]:
```
f = open('sample.txt','w')
f.write('Hello world')
f.close()

# since file is closed hence this will not work
f.write('hello')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[2], line 6
      3 f.close()
      5 # since file is closed hence this will not work
----> 6 f.write('hello')

ValueError: I/O operation on closed file.
```

### 4.write multiline strings

In [3]:
```
# we have to add multiple lines

f = open('sample1.txt','w')
f.write('hello world')
f.write('\nhow are you?')
f.close()
```

### 5.Writing to a file - TXT file .Existing file

If you try to write over an existing file, all the old content gets removed and the new content replaces it. (Remember)

In [11]:
```
# case 2 - if the file is already present

f = open('sample.txt','w')
f.write('salman khan')
f.close()
```

### 6. how exactly open() works?

### 7. Problem with w mode

In [5]:
```
# introducing append mode (append mode is solution)
f = open('sample1.txt','a')
f.write('\nI am fine')
f.close()
```

### 8. write many lines

In [6]:
```
L = ['hello\n','hi\n','how are you\n','I am fine\n']

f = open('sample.txt','w')
f.writelines(L)
f.close()
```

### 9.reading from files

In [7]:
```
# -> using read()
f = open('sample.txt','r')
s = f.read()
print(s)
f.close()
```

```
hello
hi
how are you
I am fine
```

In [4]:
```
# reading upto n chars
f = open('sample.txt','r')
s = f.read(12)
print(s)
f.close()
```

```
hello
hi
how
```

### 10.Reading Files from readlines()

In [5]:
```
# readline() -> to read line by line

f = open('sample.txt','r')
print(f.readline(),end='')
print(f.readline(),end='')
f.close()
```

```
hello
hi
```

## 11.Reading big file line by line

In [8]:
```
# reading entire using readline

f = open('sample.txt','r')

while True:
    data = f.readline()

    if data == '':
        break
    else:
        print(data,end='')
f.close()
```

```
hello
hi
how are you
I am fine
```

## 12.Using Context Manager (With)

- It's a good idea to close a file after usage as it will free up the resources
- If we dont close it, garbage collector would close it
- with keyword closes the file as soon as the usage is over

In [9]:
```
# with

with open('sample1.txt','w') as f:
    f.write('selmon bhai')
```

In [11]:
```
 # This will not work coz f.close automatically trigger when u use with
f.write('hello')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[11], line 2
      1 # This will not work coz f.close automatically trigger when u use with
----> 2 f.write('hello')

ValueError: I/O operation on closed file.
```

In [12]:
```
# try f.read() now

with open('sample.txt','r') as f:
    print(f.readline())
```

```
hello
```

## 13.reading 10 char at a time till end file

In [14]:
```
# moving within a file -> 10 char then 10 char
with open('sample.txt','r') as f:
    print(f.read(10))
    print(f.read(10))
    print(f.read(10))
    print(f.read(10))
```

```
hello
hi
h
ow are you

I am fine
```

## 14. Reading big file in chunks

In [28]:
```
# benefit? -> to load a big file in memory

big_L = ['hello world' for i in range(200)]

with open('big.txt','w') as f:
    f.writelines(big_L)
```

In [32]:
```
with open('big.txt','r') as f:
    chunk_size=10
    data = f.read(chunk_size)
    while len(data)>0:
        print(data,end='****')
        data = f.read(chunk_size)
```

```
hello worl****dhello wor****ldhello wo****rldhello w****orldhello ****worldhello**** worl
dhell****o worldhel****lo worldhe****llo worldh****ello world****hello worl****dhello wor
****ldhello wo****rldhello w****orldhello ****worldhello**** worldhell****o worldhel****l
o worldhe****llo worldh****ello world****hello worl****dhello wor****ldhello wo****rldhel
lo w****orldhello ****worldhello**** worldhell****o worldhel****lo worldhe****llo worldh*
***ello world****hello worl****dhello wor****ldhello wo****rldhello w****orldhello ****wo
rldhello**** worldhell****o worldhel****lo worldhe****llo worldh****ello world****hello w
orl****dhello wor****ldhello wo****rldhello w****orldhello ****worldhello**** worldhell**
**o worldhel****lo worldhe****llo worldh****ello world****hello worl****dhello wor****ldh
ello wo****rldhello w****orldhello ****worldhello**** worldhell****o worldhel****lo world
he****llo worldh****ello world****hello worl****dhello wor****ldhello wo****rldhello w***
*orldhello ****worldhello**** worldhell****o worldhel****lo worldhe****llo worldh****ello
world****hello worl****dhello wor****ldhello wo****rldhello w****orldhello ****worldhello
**** worldhell****o worldhel****lo worldhe****llo worldh****ello world****hello worl****d
hello wor****ldhello wo****rldhello w****orldhello ****worldhello**** worldhell****o worl
dhel****lo worldhe****llo worldh****ello world****hello worl****dhello wor****ldhello wo*
***rldhello w****orldhello ****worldhello**** worldhell****o worldhel****lo worldhe****ll
o worldh****ello world****hello worl****dhello wor****ldhello wo****rldhello w****orldhel
lo ****worldhello**** worldhell****o worldhel****lo worldhe****llo worldh****ello world**
**hello worl****dhello wor****ldhello wo****rldhello w****orldhello ****worldhello**** wo
rldhell****o worldhel****lo worldhe****llo worldh****ello world****hello worl****dhello w
or****ldhello wo****rldhello w****orldhello ****worldhello**** worldhell****o worldhel***
*lo worldhe****llo worldh****ello world****hello worl****dhello wor****ldhello wo****rldh
ello w****orldhello ****worldhello**** worldhell****o worldhel****lo worldhe****llo world
h****ello world****hello worl****dhello wor****ldhello wo****rldhello w****orldhello ****
worldhello**** worldhell****o worldhel****lo worldhe****llo worldh****ello world****hello
worl****dhello wor****ldhello wo****rldhello w****orldhello ****worldhello**** worldhell*
***o worldhel****lo worldhe****llo worldh****ello world****hello worl****dhello wor****ld
hello wo****rldhello w****orldhello ****worldhello**** worldhell****o worldhel****lo worl
dhe****llo worldh****ello world****hello worl****dhello wor****ldhello wo****rldhello w**
**orldhello ****worldhello**** worldhell****o worldhel****lo worldhe****llo worldh****ell
o world****hello worl****dhello wor****ldhello wo****rldhello w****orldhello ****worldhel
lo**** worldhell****o worldhel****lo worldhe****llo worldh****ello world****hello worl***
*dhello wor****ldhello wo****rldhello w****orldhello ****worldhello**** worldhell****o wo
rldhel****lo worldhe****llo worldh****ello world****
```

In [30]:
```
with open('big.txt','r') as f:

    chunk_size = 10

    while len(f.read(chunk_size)) > 0:
        print(f.read(chunk_size),end = '***')
        f.read(chunk_size)
```

```
dhello wor***orldhello ***o worldhel***ello world***ldhello wo***worldhello***lo worldhe*
**hello worl***rldhello w*** worldhell***llo worldh***dhello wor***orldhello ***o worldhe
l***ello world***ldhello wo***worldhello***lo worldhe***hello worl***rldhello w*** worldh
ell***llo worldh***dhello wor***orldhello ***o worldhel***ello world***ldhello wo***world
hello***lo worldhe***hello worl***rldhello w*** worldhell***llo worldh***dhello wor***orl
dhello ***o worldhel***ello world***ldhello wo***worldhello***lo worldhe***hello worl***r
ldhello w*** worldhell***llo worldh***dhello wor***orldhello ***o worldhel***ello world**
*ldhello wo***worldhello***lo worldhe***hello worl***rldhello w*** worldhell***llo worldh
***dhello wor***orldhello ***o worldhel***ello world***ldhello wo***worldhello***lo world
he***hello worl***rldhello w*** worldhell***llo worldh***dhello wor***orldhello ***o worl
dhel***ello world***ldhello wo***worldhello***lo worldhe******
```

## 15. seek and tell function

In [12]:
```
 # If you want to know which character will be printed next, then use the tell() function.
#  Here, you printed 10 characters, so the next character will be the 11th one.
#  seek() is used to move the current position to another position in the file.
# It pulls the cursor from the current position and moves it to any position you want.

with open('sample.txt','r') as f:
    f.seek(15) # means tum 15 char se aage badho
    print(f.read(10))
    print(f.tell())

    print(f.read(10))
    print(f.tell())
```

```
15

15
```

In [26]:
```
# seek during write
with open('sample.txt','w') as f:
    f.write('Hello')
    f.seek(0)
    f.write('Xa')
```

## 16.Problems with working in text mode

- can't work with binary files like images
- not good for other data types like int/float/list/tuples

## 17.working with binary file

In [10]:
```
with open('Screenshot.png','r') as f:
    f.read()

# Basically,textual data consists of Unicode characters, and binary data is different.

# They are bytes. You provided bytes, but it is searching for Unicode in the charmap,
# that’s why the error occurs.
```

```
---------------------------------------------------------------------------
UnicodeDecodeError                        Traceback (most recent call last)
Cell In[10], line 2
      1 with open('Screenshot.png','r') as f:
----> 2     f.read()

File ~\Anaconda\Lib\encodings\cp1252.py:23, in IncrementalDecoder.decode(self, input, fin
al)
     22 def decode(self, input, final=False):
---> 23     return codecs.charmap_decode(input,self.errors,decoding_table)[0]

UnicodeDecodeError: 'charmap' codec can't decode byte 0x8f in position 57: character maps
to <undefined>
```

In [28]:
```
# working with binary file

with open ('Screenshot.png','rb') as f: # readbinary used when we do binary
    with open('Screenshot_copy.png','wb') as wf:
        wf.write(f.read())
```

In [ ]:
```
# working with a big binary file
```

## 18.working with other data types

In [29]:
```
# This will through an error
# Reason - you cannot write any other datatype it has to be always a string

with open('sample.txt','w') as f:
    f.write(5)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[29], line 4
      1 # working with other data types
      3 with open('sample.txt','w') as f:
----> 4     f.write(5)

TypeError: write() argument must be str, not int
```

In [30]:
```
# correct code -> you have to provide a string

with open('sample.txt','w') as f:
    f.write('5')
```

In [8]:
```
# This will also through an error
# Reason - integer does not work with string

with open('sample.txt','r') as f: ## Look at this
    print(int(f.read()) + '5')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[8], line 5
      1 # This will also through an error
      2 # Reason - integer does not work with string
      4 with open('sample.txt','r') as f: ## Look at this
----> 5     print(int(f.read()) + '5')

ValueError: invalid literal for int() with base 10: 'hello\nhi\nhow are you\nI am fine\n'
```

In [40]:
```
# more complex data

d = {
    'name':'nitish',
    'age': 33,
    'gender':'male'
}

with open('sample.txt','w') as f:
    f.write(str(d))
```

In [43]:
```
with open('sample.txt','r') as f:
    print(f.read())
    print(type(f.read()))
```

```
{'name': 'nitish', 'age': 33, 'gender': 'male'}
<class 'str'>
```

In [45]:
```
# lets convert into dict
with open('sample.txt','r') as f:
    print(dict(f.read()))

# It throws an error because converting a string directly into a dictionary is not possibl
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[45], line 3
      1 # lets convert into dict
      2 with open('sample.txt','r') as f:
----> 3     print(dict(f.read()))

ValueError: dictionary update sequence element #0 has length 1; 2 is required
```

## 19.Serializing and Deserializing

- **Serialization** - process of converting python data types to JSON format
- **Deserialization** - process of converting JSON to python data types

**What is json**

## 20. serialization using json module

In [2]:
```
# list

import json

L = [1,2,3,4]

with open('demo.json','w') as f:
    json.dump(L,f)
```

In [4]:
```
# dict

d = {
    'name':'Kamran',
    'age':26,
    'gender':'male'
}

with open('demo.json','w') as f:
    json.dump(d,f,indent=4)
```

## 21.deserialization using JSON module

In [8]:
```
# deserialization

import json

with open('demo.json','r') as f:
    d = json.load(f)
    print(d)
    print(type(d))
```

```
{'name': 'Kamran', 'age': 26, 'gender': 'male'}
<class 'dict'>
```

## 22.serialize and deserialize using tuple

In [9]:
```

import json

t = (1,2,3,4,5)

with open('demo.json','w') as f:
    json.dump(t,f)
```

## 23.serialize and deserialize a nested dict

In [10]:
```

d = {
    'student':'nitish',
     'marks':[23,14,34,45,56]
}

with open('demo.json','w') as f:
    json.dump(d,f)
```

## 24.Serializing and Deserializing custom objects

In [13]:
```
class Person:
    def __init__(self,fname,lname,age,gender):
        self.fname = fname
        self.lname = lname
        self.age = age
        self.gender = gender


#  format to printed in
# -> Nitish Singh age -> 33 gender -> male
```

In [14]:
```
person = Person('Kamran','Khan',26,'male')
```

In [16]:
```
# As a string

import json

def show_object(person):
    if isinstance(person,Person):
        return "{} {} age -> {} gender -> {}".format(person.fname,person.lname,person.age,

with open('demo.json','w') as f:
    json.dump(person,f,default = show_object)
```

In [17]:
```
# As a dict

import json

def show_object(person):
    if isinstance(person,Person):
        return {'name':person.fname + ' ' + person.lname,'age':person.age,'gender':person.

with open('demo.json','w') as f:
    json.dump(person,f,default=show_object,indent=4)
```

In [ ]:
```
# indent arrtribute
# As a dict
```

In [18]:
```
# deserializing

import json

with open('demo.json','r') as f:
    d = json.load(f)
    print(d)
    print(type(d))
```

```
{'name': 'Kamran Khan', 'age': 26, 'gender': 'male'}
<class 'dict'>
```

## 26.Pickling

`Pickling` is the process whereby a Python object hierarchy is converted into a byte stream, and `unpickling` is the inverse operation, whereby a byte stream (from a binary file or bytes-like object) is converted back into an object hierarchy.

In [19]:
```
class Person:

    def __init__(self,name,age):
        self.name = name
        self.age = age

    def display_info(self):
        print('Hi my name is',self.name,'and I am ',self.age,'years old')
```

In [23]:
```
p = Person('kamran',26)
```

In [24]:
```
# pickle dump

import pickle
with open('person.pkl','wb') as f:
    pickle.dump(p,f)
```

In [25]:
```
# pickle load

import pickle
with open('person.pkl','rb') as f:
    p = pickle.load(f)

p.display_info()
```

```
Hi my name is kamran and I am  26 years old
```

## Pickle Vs Json

- Pickle lets the user to store data in binary format.
- JSON lets the user store data in a human-readable text format.

In [ ]:
```

```